

Run instructions

The build is all in the github repo, where the code can be run with the following code:

- Run *MTG client* with verbose **disabled**:
 - `python MTGclient.py`
- Run *MTG client* with verbose **enabled**:
 - `python MTGclient.py -v`; or
`python MTGclient.py --verbose`
- Run *MTG server* with verbose **disabled**:
 - `python MTGserver.py`
- Run *MTG server* with verbose **enabled**:
 - `python MTGserver.py -v`; or
`python MTGserver.py --verbose`

Groupings

Task / Feature	CLYDE KENDALL GO	LANCE JERWIN TAN	JOHN LAWRENCE LANDICHO	PAUL EIMRON ORTIGA
TCP Server: connection handling, framing, dispatch	Implemented connection between server and clients			
Game lifecycle: LOBBY, GAME_SETUP, MULLIGAN logic	Implemented general code for Lobby and Game Setup			Implemented Mulligan logic
Turn & phase engine (all phases/steps, transitions)		Implemented part of the phase engine		Implemented game transitions
Priority & Stack logic, spell/ability resolution			Implemented stack resolution	
Combat system (attackers, blockers, damage)		Implemented a major part of	Implemented part of the combat phase	Implemented part of the combat phase

Task / Feature	CLYDE KENDALL GO	LANCE JERWIN TAN	JOHN LAWRENCE LANDICHO	PAUL EIMRON ORTIGA
		the combat system		
Client implementation & state rendering			Implemented client-side command updates	
PDU serialisation/deserialisation (all 25 PDU types)	Implemented this feature partially		Implemented this feature partially	Implemented this feature partially
Error handling, PING/PONG heartbeat, disconnect logic	Implemented PING/PONG heartbeat and disconnect logic			
Verbose mode (client + server PDU logging, toggle on/off)	Implemented verbose mode			
Testing & interoperability		Tested and debugged parts of codes		
README / documentation / AI disclosure	Created the documentation			

AI Usage Policy

Tools Used: ChatGPT, Google Gemini

- **How the AI was used:**

- To check the possible solution for errors encountered and how to handle the bug fixes.
- Used for in depth explanation and guide to help the group develop the application-based Magic the Gathering game.
- To provide sample or simple code for reference as to how to implement the features of the game based on the specifications.

- **Human verification:**

- The group ensures that all code suggestions generated by ChatGPT or Gemini were manually reviewed and edited to fit the specifications of the project.

- The implementation was tested by the group using local client-server execution across multiple terminal instances to debug, test, and verify some edge cases.

Known Limitations & Deviations from RFC

- Based on the specifications, the decks are limited to 1–50 cards in validation checks, whereas the official Magic the Gathering game formats typically require a 60-card minimum.
- Each card's effects were simplified into direct blocker-to-attacker mappings rather than a fully layered MTG comprehensive rules engine.
- The priority feature follows a basic and simplified 2-pass sequence to resolve stack items or transition phases.
- The server strictly enforces a maximum of two (2) concurrent client sockets in the lobby state, and does not support multi-table matches or spectator sockets.
- The server enforces a hard 60-second priority timer per turn without dynamic pause, resume, or extension protocols.